KU Matching for Data Science

Transfer Credit Match — Data Science Branch · Proof of Concept · May 2026

A structured, intelligent system that helps transferring students at Chicago-area colleges identify course equivalencies using Knowledge Unit (KU) mappings, a PDF transcript parser, and a standardized academic data pipeline.

Presented by Data Science team:

Jordi Banuelos
Henal Parikh
Hamza Syed
Om Patel
Lakshay Vivek

Overview: Bridging Institutions Through Academic Data

Students who transfer between Chicago-area colleges often face a critical question: **which of my completed courses will count at my new school?** The Transfer Credit Match system answers that question using structured academic data, Knowledge Unit (KU) mappings, and an automated PDF transcript parser.

The Problem

Transfer students lose credits when course equivalencies are unclear or evaluated manually.

Our Solution

A KU-based matching engine that compares course competency profiles across institutions.

Our Branch

Data Science layer: PDF parsing, KU taxonomy, course mapping, and data standardization.

Solution Overview

The Data Science branch is organized into three logical areas: the **transcript parser**, the **CSV seed data**, and the **readme documentation**. Together they form the data backbone of the matching system.

TranscriptPDF_reader/

- `pdf_reader.py` — Parses Banner and web-saved PDF transcripts; extracts course codes, names, grades, and credits

Root

- `readme.txt` — Branch documentation

csv_exports/ — 11 CSV Files

- `institutions.csv` — 4 Chicago-area schools
- `programs.csv` — 9 academic programs
- `courses.csv` — 25 seeded courses
- `knowledge_units.csv` — 26 CAE-aligned KUs
- `course_ku.csv` — 32 course-KU links
- `students.csv` — Enrollment records
- `transfer_requests.csv` — Student requests
- `course_match.csv` — Matched course pairs
- `match_history.csv` — Audit log
- `admins.csv` / `directors.csv` / `users.csv`

System Architecture

The proof of concept seeds **four real Chicago-area institutions** offering a combined **nine academic programs** across Computer Science, Cybersecurity, IT, and related fields. This diversity creates realistic cross-institution matching scenarios.

Data Ingestion

Computer Science · Cybersecurity · Cyber Security & Info Assurance

j

Information Technology · Software Development

Binary Classification Model

Computer Forensics · Computer Science

University of Chicago

Cloud Computing · Information Technology

Data ingestion

Knowledge Units (KUs) are standardized academic competency areas aligned with the **CAE (Center for Academic Excellence) cybersecurity framework**. Each course maps to one or more KUs, enabling semantic comparison across institutions regardless of course title differences.

Core Computing KUs

- Programming Concepts
- Basic Scripting & Programming
- Low Level Programming
- Operating Systems Concepts
- Operating Systems Theory
- Database Management

Cybersecurity KUs

- Cybersecurity Basics & Foundations
- Network Defense
- Intrusion Detection / Prevention
- Web Application Security
- Vulnerability Analysis
- Basic Cryptography
- Cybersecurity Ethics

26

Total KUs Seeded

CAE-aligned competency areas covering programming, networking, security, and systems

32

Course-KU Links

Many-to-many mappings connecting courses to their competency profiles

25

Courses Seeded

Sample catalog spanning all four institutions and nine programs

Binary Classification Model

Courses are mapped to multiple KUs in a **many-to-many relationship**. When a transfer request is submitted, the system compares the KU profiles of the source and target courses. **Sufficient KU overlap = a recommended match.**

Course ID	Course Name	Mapped KU IDs
1	Data Structures & Algorithms	KU 1 — Programming Concepts
2	Network Security	KU 2 — Cybersecurity Basics
13	Operating Systems	KU 7, 12, 14, 24
17	Database Systems	KU 16, 17
23	Intro to Computer Security	KU 5, 6, 11, 15, 19, 25, 26
24	Internet Security	KU 10, 21, 24, 26

Course 23 (Intro to Computer Security) maps to **7 different KUs** — the most in the dataset — reflecting its broad, cross-cutting cybersecurity scope. This richness enables more accurate cross-institution equivalency detection.

Results / Output

The system includes a fully seeded end-to-end example demonstrating the complete workflow from student enrollment through director review.

The Student

Student ID: 1
Enrolled at: Roosevelt University
Program: Computer Science

The Transfer Request

From: Roosevelt → Harold Washington College
Source Course: CS101 — Data Structures & Algorithms (4 cr)
Target Course: SD102 — Web Development (3 cr)
Status: Pending

The Review Process

The KU profiles of CS101 and SD102 are compared. If overlap is sufficient, the match is recommended. A program director at Harold Washington College reviews and finalizes the decision.

Example Walkthrough

File: TranscriptPDF_reader/pdf_reader.py

Parses student PDF transcripts to extract course codes, names, grades, and credit hours — auto-populating transfer requests and eliminating manual entry errors.

Usage

```
python pdf_reader.py --pdf <path_to_transcript.pdf>
```

Supported Formats

- Banner-generated PDF transcripts
- Web-saved / printed PDFs (with layout noise handling)

Libraries

- `pdfplumber` — table extraction
- `re` — regex fallback parsing
- `pandas` — data structuring & export

Why Parsing Matters

Manual transcript entry is slow and error-prone. The parser extracts structured course data directly from PDFs, feeding it into the matching pipeline with minimal human intervention.

Extract

Read PDF tables and text using pdfplumber; fall back to regex for non-standard layouts

Structure

Organize extracted rows into a pandas DataFrame with standardized column names

Submit

Export parsed data to populate transfer request records in the database

Data Science Contribution

Extracted transcript data is rarely clean. Course codes may be missing, titles may use different naming conventions, and KU terminology varies across institutions. This layer ensures data entering the matching pipeline is **consistent, complete, and trustworthy**.

KU Normalization

Maps variant labels like *"binary arithmetic"* and *"base conversion"* to a single canonical KU label, ensuring consistent matching regardless of institutional terminology.

Parser Output Validation

Checks every parsed row for missing course codes, titles, or credits. Flags malformed records before they enter the matching pipeline.

Benchmark Dataset Creation

Generates synthetic and sample records so the team can test the matching pipeline before large volumes of real data are available.

In short: The parser *extracts* the data → this layer *cleans and validates* it → the matching system uses *higher-quality input* as a result.

Challenges

The pipeline flows from raw PDF transcript to a finalized, audited match decision — with the KU Mapping Engine at its core and the Data Standardization Layer ensuring input quality at every stage.

What We Built

1. **Data Modeling & CSV Seed Data**

   Designed and populated all 11 CSV files with realistic academic records across 4 institutions

2. **PDF Transcript Parsing Pipeline**

   Built pdf_reader.py to extract and structure course data from Banner and web-saved PDFs

3. **KU Taxonomy & Course Mapping**

   Designed 26 CAE-aligned Knowledge Units and mapped 25 courses across 32 many-to-many links

Key Outcomes

- ✅ **Working POC**

  Fully seeded end-to-end transfer request example

- ✅ **Scalable Design**

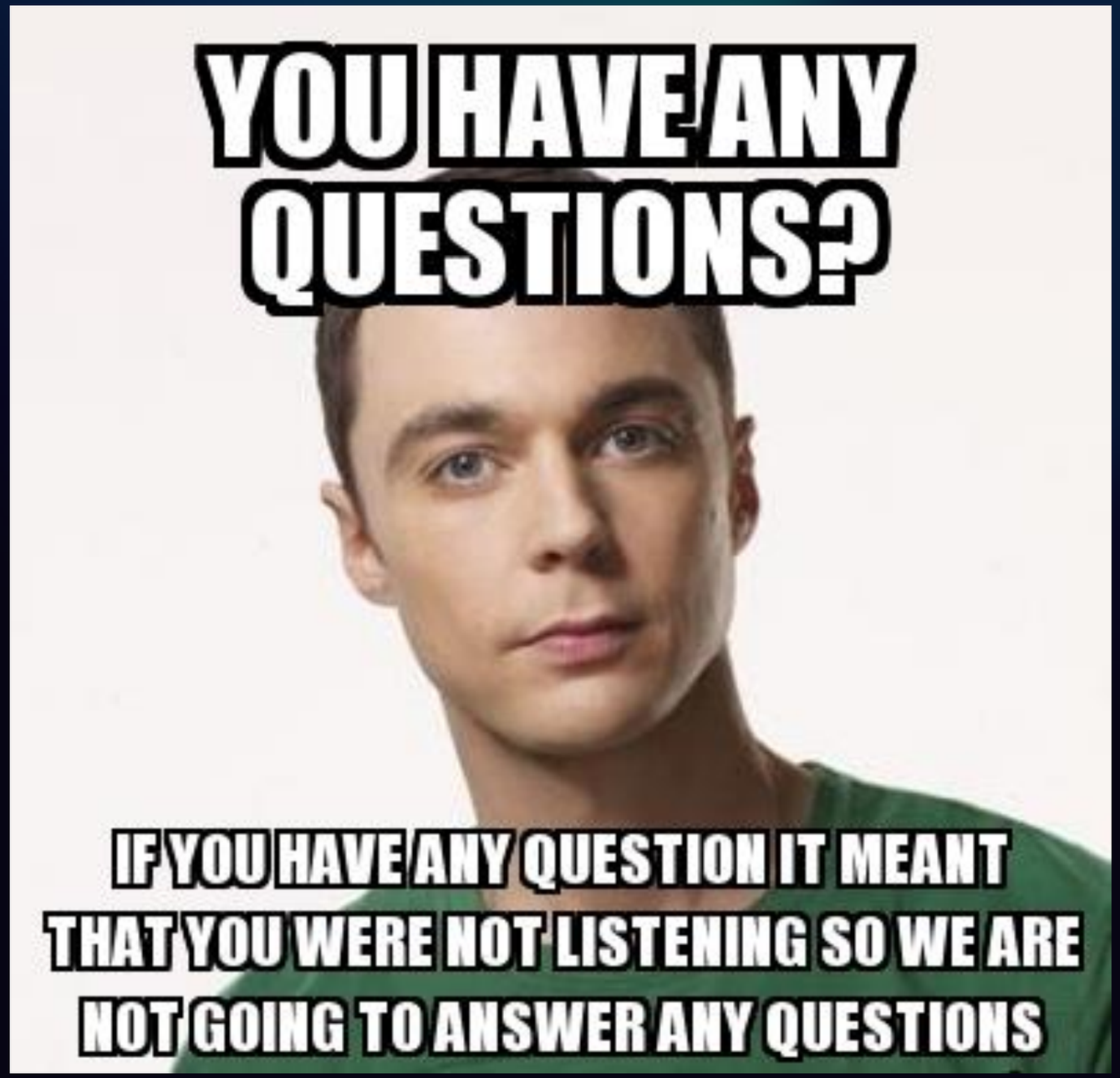
  Architecture supports adding more institutions and KUs

- ✅ **Audit Trail**

  match_history.csv preserves every decision for accountability

Future Work

Conclusion


YOU HAVE ANY QUESTIONS?
IF YOU HAVE ANY QUESTION IT MEANT THAT YOU WERE NOT LISTENING SO WE ARE NOT GOING TO ANSWER ANY QUESTIONS